

Automatisierter Sicherheitsbericht

Erstellt am: 18.05.2026 14:09:06

1. Management-Zusammenfassung

Im vorliegenden automatisierten Scan der Anwendung `http://juice-shop:3000` wurden mehrere Seiten einer Single-Page-Application (SPA) untersucht. Dabei wurden drei tatsächlich ausgewiesene Schwachstellen gefunden, davon zwei mit hohem bis kritischem Risiko:

1. SQL Injection mit Login-Bypass auf der Login-Seite

Diese Schwachstelle ist die kritischste Feststellung. Laut Rohdaten konnte mit dem Eingabewert ` ' OR 1=1 -- ` ein erfolgreicher Login erzwungen werden. Zusätzlich wurde ein JWT-Token für ein Admin-Konto erfasst. Das weist auf einen unmittelbaren unautorisierten Zugriff mit weitreichenden Rechten hin.

2. Cross-Site Scripting (XSS) auf der Suchfunktion

In der Suchseite wurde ein Script-Payload erfolgreich ausgeführt ("Alert message is triggered at input"). Damit könnten Angreifer schädlichen Code im Browser anderer Nutzer ausführen.

3. Fehlender Schutz gegen Brute-Force-Angriffe im Login

Nach 40 fehlgeschlagenen Login-Versuchen wurde laut Scan keine wirksame Schutzmaßnahme erkannt. Das erhöht das Risiko von Passwort-Angriffen deutlich.

Weitere untersuchte Seiten enthielten Formulare oder Eingabefelder, aber keine zusätzlich bestätigten Schwachstellen in den Rohdaten.

Gesamtbewertung:

Das Risiko ist hoch bis kritisch, weil bereits ein erfolgreicher Admin-Zugriff per SQL Injection nachgewiesen wurde. Diese Schwachstelle sollte sofort behoben werden. Danach sollten XSS und fehlende Login-Schutzmechanismen priorisiert abgearbeitet werden.

2. Technischer Überblick über die analysierten Seiten

Der Scanlauf war laut RUN-Metadaten zum Zeitpunkt der Erfassung noch als `running` markiert. Aus den gelieferten TARGET-Blöcken lassen sich folgende analysierte URLs ableiten:

- `http://juice-shop:3000/#`
- `http://juice-shop:3000/#login`
- `http://juice-shop:3000/#contact`

- `http://juice-shop:3000/#!/about`
- `http://juice-shop:3000/#!/photo-wall`
- `http://juice-shop:3000/#!/complain`
- `http://juice-shop:3000/#!/chatbot`

Beobachtete Funktionen und Eingabeoberflächen

Start-/Suchseite

- Auf der Startseite wurde eine XSS-Schwachstelle über den Suchparameter festgestellt:
- `/#!/search?q=...`

Login-Seite

- Erkanntes Login-Formular mit:
- Feld `email`
- Feld `password`
- Checkbox `Remember me`
- Bestätigte Schwachstellen:
- SQL Injection / Login-Bypass
- fehlende Brute-Force-Schutzmaßnahmen

Kontakt-Seite

- Formularähnliche SPA-Komponente mit:
- Author
- Comment
- Range-Feld
- CAPTCHA-Ergebnis
- Kein bestätigter Befund in "Vulnerabilities"
- Doppelte Formularerkennung vorhanden; inhaltlich identisch, daher als Duplikat zu werten

Beschwerde-Seite

- Formular mit:
- Customer
- Message
- Datei-Upload ("Invoice")
- Kein bestätigter Schwachstellenfund in den Rohdaten

Chatbot-Seite

- Eingabefeld `message`

- Kein bestätigter Schwachstellenfund

Header- und Konfigurationshinweise

Mehrere Antworten enthielten unter anderem:

- `access-control-allow-origin: \*`
- `x-content-type-options: nosniff`
- `x-frame-options: SAMEORIGIN`

Der Header `access-control-allow-origin: \*` kann je nach Architektur sicherheitsrelevant sein. Aus den vorliegenden Rohdaten allein lässt sich jedoch keine konkrete Schwachstelle ableiten, da weder sensible Cross-Origin-Antworten noch missbräuchliche CORS-Szenarien belegt sind. Daher wird dies nicht als bestätigte Schwachstelle gewertet.

3. Priorisierte Liste der gefundenen Schwachstellen

1. SQL Injection mit Admin-Login-Bypass

- Betroffene URL: `http://juice-shop:3000/#/login`
- Risiko: Kritisch
- Priorität: Sofort
- Warum kritisch: Erfolgreiche Authentifizierungsumgehung wurde belegt, inklusive JWT für ein Admin-Konto.

2. Reflected/DOM-nahe XSS in der Suchfunktion

- Betroffene URL: `http://juice-shop:3000/#/search?q=...`
- Risiko: Hoch
- Priorität: Sehr hoch
- Warum relevant: Beliebiger Script-/JavaScript-Inhalt wurde im Browser ausgeführt.

3. Fehlender Brute-Force-Schutz im Login

- Betroffene URL: `http://juice-shop:3000/#/login`
- Risiko: Mittel
- Priorität: Hoch
- Warum relevant: Erlaubt systematische Passwort-Raterversuche und begünstigt Kontoübernahmen.

4. Detailanalyse pro Schwachstelle

Schwachstelle 1: SQL Injection mit Login-Bypass

- Betroffene Seite / URL:

`http://juice-shop:3000/#/login`

In den Rohdaten ist der Vulnerability-Eintrag teils mit `http://juice-shop:3000/#/search` referenziert, der Kontext aus Formular, Feldname und Credential-Erfassung weist jedoch klar auf die Login-Funktion hin.

- Typ der Schwachstelle:

SQL Injection / Authentifizierungsumgehung

- Technische Beobachtung:

Im Login-Formular wurde im Feld `email` der Payload

` OR 1=1 --`

verwendet. Der Scanner vermerkt ausdrücklich:

"Login bypass via SQL injection".

Zusätzlich wurde unter `credentials` ein erfolgreicher Login mit JWT-Token dokumentiert. Das Token enthält Hinweise auf ein Konto mit Rolle admin (`"role":"admin"`).

- Mögliche Auswirkungen:

- Umgehung der Anmeldung ohne gültige Zugangsdaten
- Zugriff auf Benutzerkonten, hier mutmaßlich sogar Administratorzugriff
- Datenmanipulation oder Datenabfluss
- Vollständige Kompromittierung geschützter Funktionen
- Je nach Backend potenziell weitere Datenbankangriffe

- Geschätzte CVSS 4.0-Bewertung:

9.8 / 10 (Kritisch)

Begründung:

- Netzwerkbasiert ausnutzbar
- Keine gültige Vor-Authentisierung erforderlich
- Niedrige Angriffskomplexität
- Direkter Einfluss auf Vertraulichkeit, Integrität und wahrscheinlich auch Verfügbarkeit geschützter Funktionen

Einschränkung: Eine exakte CVSS-4.0-Vektorbildung ist anhand der Rohdaten nicht vollständig belastbar, da Backend-Struktur und Reichweite des Datenbankzugriffs unbekannt sind. Wegen des

nachgewiesenen Admin-Bypasses ist eine sehr hohe Bewertung jedoch fachlich gerechtfertigt.

- Priorität:

Sofort

- Empfohlene Gegenmaßnahmen:

- Sämtliche Datenbankabfragen auf parametrisierte Queries / Prepared Statements umstellen
- Serverseitige Eingaben strikt validieren
- Authentifizierungslogik auf Injection-Sicherheit prüfen
- Logging und Monitoring auf missbräuchliche Login-Muster aktivieren
- Alle betroffenen Sessions/Tokens invalidieren
- Prüfen, ob bereits unautorisierte Zugriffe stattgefunden haben
- Code-Review und gezielte Nachtests für alle datenbanknahen Eingaben

- Leicht verständliche Erklärung:

Das Login kann offenbar mit einem speziell formulierten Texteingabewert ausgetrickst werden. Statt ein echtes Passwort zu kennen, kann sich ein Angreifer so als Benutzer anmelden - nach den Rohdaten sogar als Administrator. Das ist vergleichbar damit, wenn ein Türschloss sich mit einem falsch geformten Schlüssel einfach öffnen lässt.

Schwachstelle 2: Cross-Site Scripting (XSS) in der Suchfunktion

- Betroffene Seite / URL:

`http://juice-shop:3000/#/search?q=%3Ciframe%20src%3D%22javascript:alert(%60xss%60)%22%3E`

- Typ der Schwachstelle:

Cross-Site Scripting (XSS)

- Technische Beobachtung:

Der Scanner meldet:

- Typ: `XSS Injection`

- Eingabefeld: `search-input`

- Payload: `
&amp;amp;lt;/div&amp;amp;gt;
&amp;amp;lt;div data-bbox="71 828 501 846" data-label="Text"&amp;amp;gt;
&amp;amp;lt;p&amp;amp;gt;- Beobachtung: "Alert message is triggered at input"&amp;amp;lt;/p&amp;amp;gt;
&amp;amp;lt;/div&amp;amp;gt;
&amp;amp;lt;div data-bbox="71 865 935 930" data-label="Text"&amp;amp;gt;
&amp;amp;lt;p&amp;amp;gt;Daraus lässt sich ableiten, dass nicht vertrauenswürdige Eingaben in der Suchfunktion so verarbeitet wurden, dass JavaScript im Browser ausgeführt werden konnte. Aufgrund der SPA-URL-Struktur ist ein DOM-basiertes oder reflektiertes XSS-Szenario plausibel; die Rohdaten erlauben jedoch keine&amp;amp;lt;/p&amp;amp;gt;
&amp;amp;lt;/div&amp;amp;gt;
&amp;amp;lt;div data-bbox="471 961 524 976" data-label="Page-Footer"&amp;amp;gt;Seite 5&amp;amp;lt;/div&amp;amp;gt;

trennscharfe endgültige Einordnung.

- Mögliche Auswirkungen:
- Diebstahl von Sitzungsinformationen oder Tokens
- Ausführung von Aktionen im Namen eingeloggter Nutzer
- Anzeige manipulierter Inhalte
- Weiterleitung auf schädliche Seiten
- Phishing innerhalb der legitimen Anwendung

- Geschätzte CVSS 4.0-Bewertung:

8.1 / 10 (Hoch)

Begründung:

- Netzwerkbasiert
- Geringe Angriffskomplexität
- Für praktische Ausnutzung ist typischerweise eine Benutzerinteraktion erforderlich (z. B. Aufruf eines manipulierten Links)
- Potenziell erheblicher Einfluss auf Vertraulichkeit und Integrität im Browserkontext

Einschränkung: Unklar ist, ob Sicherheitsmechanismen wie CSP aktiv sind oder ob ein Session-Kontext tatsächlich abgreifbar ist. Deshalb vorsichtige, aber hohe Bewertung.

- Priorität:

Sehr hoch

- Empfohlene Gegenmaßnahmen:

- Alle Suchparameter kontextgerecht escapen/encoden
- Keine ungeprüfte Einfügung von Nutzereingaben in HTML oder DOM
- Unsichere Konstrukte wie `innerHTML` vermeiden
- Wenn Client-seitig gerendert wird: nur sichere DOM-APIs verwenden
- Content Security Policy (CSP) einführen bzw. verschärfen
- Sicherheits-Tests speziell für DOM-XSS ergänzen

- Leicht verständliche Erklärung:

In der Suchfunktion kann offenbar schädlicher Code so eingegeben werden, dass er im Browser ausgeführt wird. Ein Angreifer könnte dadurch anderen Nutzern manipulierte Inhalte zeigen oder Informationen aus deren Sitzung abgreifen. Vereinfacht gesagt: Das Suchfeld behandelt fremde Eingaben nicht nur als Text, sondern teilweise wie ausführbare Anweisungen.

Schwachstelle 3: Fehlender Schutz gegen Brute-Force-Angriffe

- Betroffene Seite / URL:

`http://juice-shop:3000/#!/login`

- Typ der Schwachstelle:

Fehlende oder unzureichende Brute-Force-Protection

- Technische Beobachtung:

Der Scanner dokumentiert:

"Keine Änderung von Statuscode, Response-Body oder Retry-After nach 40 fehlgeschlagenen Login-Versuchen"

Das deutet darauf hin, dass nach wiederholten Fehlversuchen weder Sperren, Verzögerungen, Captcha-Verschärfungen noch erkennbare Rate-Limits aktiv wurden.

- Mögliche Auswirkungen:

- Automatisierte Passwort-Raterversuche gegen Benutzerkonten
- Erhöhtes Risiko erfolgreicher Kontoübernahmen bei schwachen Passwörtern
- Erleichterung von Credential-Stuffing-Angriffen
- Belastung des Authentifizierungssystems

- Geschätzte CVSS 4.0-Bewertung:

6.9 / 10 (Mittel)

Begründung:

- Netzwerkbasiert ausnutzbar
 - Keine besonderen Voraussetzungen
 - Erfolg hängt von Passwortstärke, vorhandenen Benutzerkonten und weiteren Schutzmaßnahmen ab
- Einschränkung: Da keine Informationen über MFA, Passwortpolitik oder nachgelagerte Erkennungssysteme vorliegen, ist eine konservative mittlere Bewertung angemessen.

- Priorität:

Hoch

- Empfohlene Gegenmaßnahmen:

- Rate-Limiting pro IP, Konto und Gerätefingerabdruck
- Progressive Verzögerungen nach Fehlversuchen
- Temporäre Sperren oder zusätzliche Verifikation nach Schwellwerten
- MFA für privilegierte und möglichst alle Benutzerkonten
- Erkennung von Credential-Stuffing-Mustern

- Überwachung und Alarmierung bei auffälligen Login-Serien

- Leicht verständliche Erklärung:

Die Anmeldung lässt anscheinend sehr viele Fehlversuche zu, ohne spürbar zu bremsen oder zu blockieren. Dadurch können Angreifer massenhaft Passwortkombinationen ausprobieren, bis sie zufällig Erfolg haben.

5. Konkrete Handlungsempfehlungen

Sofortmaßnahmen

1. SQL-Injection im Login umgehend beheben

- Authentifizierungsabfragen prüfen und auf parametrisierte Datenbankzugriffe umstellen
- Produktionszugänge und Admin-Konten auf Missbrauch kontrollieren
- Bereits ausgestellte Tokens/Sessions invalidieren
- Falls möglich, Anmeldungen vorübergehend zusätzlich absichern

2. XSS in der Suche schließen

- Suchparameter strikt als Daten behandeln, nicht als HTML/Script
- Rendering-Pfade in Frontend-Komponenten prüfen
- Sicherheitsnachtest mit verschiedenen XSS-Payloads durchführen

3. Login gegen Angriffe härten

- Rate-Limits, Kontoschutz und MFA implementieren
- Monitoring für Serien fehlgeschlagener Anmeldungen aktivieren

Kurzfristige Verbesserungen

4. Gezielte Code-Reviews

- Besonders für:
- Login-Logik
- Suchfunktion
- alle Formulare mit Freitextfeldern
- Datei-Upload auf `/#/complain`

5. Sicherheits-Tests ausweiten

- Formularfelder auf serverseitige Validierung prüfen
- DOM-basierte Schwachstellen in der SPA gesondert testen
- Upload-Funktion auf Typprüfung, Inhaltsprüfung und sichere Speicherung prüfen

6. Härtung der Browser-Sicherheitsmechanismen

- CSP einführen oder verbessern
- Prüfung weiterer Security Header
- CORS-Konfiguration fachlich bewerten, insbesondere wegen `access-control-allow-origin: \*`

Organisatorische Maßnahmen

7. Incident-Review

- Prüfen, ob der gefundene Login-Bypass bereits ausgenutzt wurde
- Logs auf verdächtige Admin-Anmeldungen oder ungewöhnliche Tokens untersuchen

8. Secure Development Practices

- Standardisierte Eingabevalidierung
- Pflicht für Prepared Statements
- Sicherheitsprüfungen vor Releases
- Entwickler-Schulung zu Injection- und XSS-Risiken

6. Gesamtfazit

Die ausgewerteten Rohdaten zeigen ein ernstes Sicherheitsniveau-Problem in der Anwendung. Besonders kritisch ist die nachgewiesene SQL Injection in der Login-Funktion, da sie offenbar einen Admin-Login ohne legitime Zugangsdaten ermöglicht. Diese einzelne Schwachstelle reicht bereits aus, um die Anwendung als akut gefährdet einzustufen.

Zusätzlich wurde eine XSS-Schwachstelle in der Suchfunktion bestätigt, die Angriffe auf Benutzerbrowser ermöglicht. Der ebenfalls festgestellte fehlende Brute-Force-Schutz verschärft das Risiko weiterer Kontoübernahmen.

Andere analysierte Seiten wie Kontakt, Beschwerdeformular, Chatbot, About und Photo Wall wurden zwar technisch erfasst, jedoch ohne bestätigte zusätzliche Schwachstellen in den vorliegenden Rohdaten. Das bedeutet nicht, dass diese Bereiche sicher sind, sondern nur, dass in diesem Datenauszug keine weiteren bestätigten Befunde vorliegen.

Empfohlene Priorisierung:

1. SQL Injection / Login-Bypass sofort beheben
2. XSS in der Suche kurzfristig schließen
3. Brute-Force-Schutz am Login nachziehen
4. Danach vertiefte Nachtests aller Eingabefelder und Upload-Funktionen durchführen

Wenn gewünscht, kann ich im nächsten Schritt aus diesen Ergebnissen auch noch einen formaleren Audit-Bericht mit Tabelle, Risikomatrix und CVSS-Vektor-Notation erstellen.